

PES Version Control System (VCS) Project

Overview

This project implements a simplified Git-like Version Control System (VCS) in C. It includes core components such as object storage, tree structure, staging area (index), and commit history. The system demonstrates how version control systems like Git internally manage data and track changes efficiently.

Phase 1: Object Storage

In this phase, blob objects were implemented using SHA-256 hashing. Each file is stored as an object, ensuring data integrity and deduplication.

Screenshot 1A: Object Test Output

```
abhishek@abhishek-VirtualBox:~/PES1UG24CS650-pes-vcs$ ./test_objects
stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B: Stored Objects

```
abhishek@abhishek-VirtualBox:~/PES1UG24CS650-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
```

Phase 2: Tree Structure

Tree objects were implemented to represent directory structures. Each tree object contains metadata such as file mode, filename, and corresponding object hash.

Screenshot 2A: Tree Test Output

```
abhishek@abhishek-VirtualBox:~/PES1UG24CS650-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B: Tree Object (xxd)

```
abhishek@abhishek-VirtualBox:~/PES1UG24CS650-pes-vcs$ xxd .pes/objects/dd/7493dd11a4e1855394b78721d8c44a16ff6db5efb484e209c638552d65d941 | head -20
00000000: 7472 6565 2034 3900 3130 3036 3434 2064  tree 49.100644 d
00000010: 756d 6d79 2e74 7874 0000 0000 0000 0000  ummy.txt.....
00000020: 0000 0000 0000 0000 0000 0000 0000 0000  .....
00000030: 0000 0000 0000 0000 0000 0000 0000 0000  .....
abhishek@abhishek-VirtualBox:~/PES1UG24CS650-pes-vcs$
```

Phase 3: Index (Staging Area)

The staging area was implemented using index files. Functions such as `index_load`, `index_save`, and `index_add` were developed to track staged files.

Screenshot 3A: Status Output

```
abhishek@abhishek-VirtualBox:~/PES1UG24CS650-pes-vcs$ ./pes init
echo "hello" > file1.txt
echo "world" > file2.txt

./pes add file1.txt file2.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
  staged:    hello.txt
  staged:    file1.txt
  staged:    file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked: index.c
  untracked: pes.h
  untracked: commit.c
  untracked: test_objects.c
  untracked: pes.c
  untracked: README.md
  untracked: index.h
  untracked: test_tree.c
  untracked: commit.h
  untracked: tree.c
  untracked: Makefile
  untracked: tree.h
  untracked: .gitignore
  untracked: test_sequence.sh
  untracked: object.c
```

Screenshot 3B: Index File

```
abhishek@abhishek-VirtualBox:~/PES1UG24CS650-pes-vcs$ cat .pes/index
100664 05c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43 1776320825 12 hello.txt
100664 2cf8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 1776321871 6 file1.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776321871 6 file2.txt
abhishek@abhishek-VirtualBox:~/PES1UG24CS650-pes-vcs$
```

Phase 4: Commit System

The commit system was implemented to create commits, link parent commits, and display commit history. The system stores commit metadata such as author, timestamp, and message.

Screenshot 4A: Commit Log

```
abhishek@abhishek-VirtualBox:~/PES1UG24CS650-pes-vcs$ ./pes log
commit 9facdacb97a7c98512666c543a606100e94749941092f803928225638c2e5634
Author: PES User <pes@localhost>
Date: 1776320825

    Second commit

commit 3d162e471e7e1b0ee959b520543e0b1f7e6b148f51d3232f64f750b9f6b7cd10
Author: PES User <pes@localhost>
Date: 1776320825

    Initial commit

abhishek@abhishek-VirtualBox:~/PES1UG24CS650-pes-vcs$
```

Screenshot 4B: Object Files

```
abhishek@abhishek-VirtualBox:~/PES1UG24CS650-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/3d/162e471e7e1b0ee959b520543e0b1f7e6b148f51d3232f64f750b9f6b7cd10
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/9f/acdacb97a7c98512666c543a606100e94749941092f803928225638c2e5634
.pes/objects/dd/7493dd11a4e1855394b78721d8c44a16ff6db5efb484e209c638552d65d941
.pes/refs/heads/main
abhishek@abhishek-VirtualBox:~/PES1UG24CS650-pes-vcs$
```

Screenshot 4C: HEAD and Branch

```
abhishek@abhishek-VirtualBox:~/PES1UG24CS650-pes-vcs$ cat .pes/HEAD
cat .pes/refs/heads/main
ref: refs/heads/main
9facdacb97a7c98512666c543a606100e94749941092f803928225638c2e5634
abhishek@abhishek-VirtualBox:~/PES1UG24CS650-pes-vcs$
```

Analysis Questions

Q5: Branching

5.1 What is a branch?

A branch is a pointer to a specific commit in the version control system. It allows developers to work on different features independently without affecting the main codebase.

5.2 How are branches implemented in PES?

In PES, branches are implemented as files inside `.pes/refs/heads/`. Each branch file stores the hash of the latest commit. The HEAD file points to the current branch.

5.3 How would you implement checkout?

Checkout can be implemented by updating the HEAD pointer, restoring files from the selected commit's tree, and updating the index accordingly.

Q6: Garbage Collection

6.1 Why is garbage collection needed?

Garbage collection removes unused or unreachable objects from the repository, reducing storage usage and improving efficiency.

6.2 How would you implement garbage collection?

Garbage collection can be implemented by traversing all reachable objects starting from HEAD and deleting any objects that are not referenced.

Conclusion

This project demonstrates the internal working of a version control system, including object storage, indexing, tree representation, and commit history. It provides a practical understanding of how systems like Git manage versioned data efficiently.